

Analisi What-If dell'Impatto del Refactoring sulla Propensione ai Bug

Studio empirico su Apache Bookkeeper e Apache Storm

Sofia Tosti
Ingegneria del Software 2
Università degli Studi di Roma Tor Vergata
A.A. 2024/2025

Il Problema



Il refactoring può davvero prevenire i bug?

Manutenibilità del Software

Il codice sorgente subisce modifiche frequenti e la presenza di difetti a livello di metodo compromette la stabilità del sistema

Necessità di Metriche

Serve un approccio data-driven per guidare le decisioni di refactoring e ridurre il debito tecnico

Domande di Ricerca



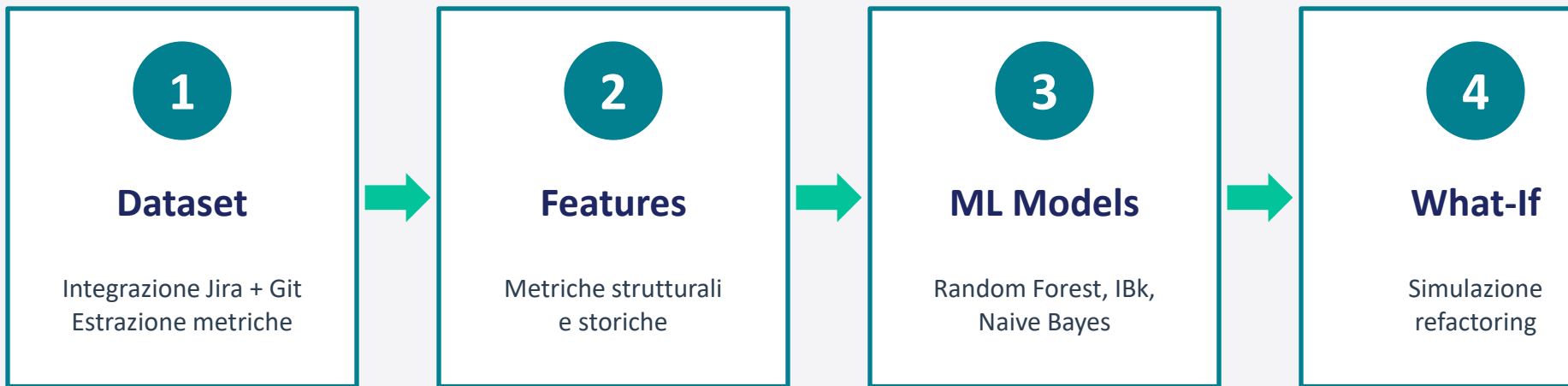
RQ1

Quale classificatore è in grado di prevedere con maggiore accuratezza la presenza di bug a livello di metodo?

RQ2

Qual è l'impatto potenziale di un intervento di refactoring mirato sulla riduzione dei metodi buggy stimati?

Metodologia



Metriche Analizzate



Metriche Strutturali

Cosa possiamo modificare

- Lines of Code (LOC)
- Complessità Ciclomantica
- Complessità Cognitiva
- Profondità Annidamento
- Numero di Parametri
- Code Smells (PMD)

Metriche Storiche

Storia evolutiva del codice

- Numero di Revisioni
- Numero di Autori
- Code Churn
 - Totale
 - Massimo
 - Medio

Etichettatura



Integrazione di Jira+Git

- Bug risolti e chiusi da Jira
- Commit correttivi da Git
- Collegamento via ticket ID

Determiniamo:

- Fixed Version (FV)
- Opening Version (OV)
- Injected Version (IV) ?

Tecnica Proportion

Per ticket con IV mancante:

$$P = (FV - IV) / (FV - OV)$$

- Ordinamento per Resolution Date
- Analisi incrementale
- P stimato come media dei valori già osservati

Impatto delle metriche

Apache Bookkeeper

Sensibile alle metriche strutturali

→ Metodi lunghi e complessi accumulano difetti

Rank	Feature	SpearmanRho
1	Revisions	0.6175
2	Authors	0.5901
3	LOC	0.1463
4	NestingDepth	0.1188
5	CycloComplexity	0.1158
6	CognitiveComplexity	0.0912
7	ParameterCount	0.0906
8	NumberOfCodeSmells	0.0648
9	TotalChurn	0.0393
10	MaxChurn	-0.0102
11	AvgChurn	-0.0843

Apache Storm

Dominato dalla storia evolutiva

→ Bug da interazioni complesse e modifiche incrementali

Rank	Feature	SpearmanRho
1	Authors	0.3922
2	Revisions	0.3820
3	TotalChurn	0.2300
4	MaxChurn	0.1723
5	AvgChurn	0.1547
6	NestingDepth	0.0545
7	CognitiveComplexity	0.0529
8	CycloComplexity	0.0492
9	LOC	0.0449
10	ParameterCount	0.0313
11	NumberOfCodeSmells	-0.024

Validazione e Preparazione dei Dati

10x10 Cross-Validation

- Dataset diviso in 10 fold
- 9 fold per training, 1 per test
- Ripetuto 10 volte

Beneficio: **valutazione robusta** e riduzione overfitting

Bilanciamento delle Classi

Problema: classi sbilanciate (più metodi non-buggy che buggy)

Bookkeeper → SMOTE (+3,144)

Storm → Down-sampling (20K)



Feature Selection: Information Gain

Misura quanto ogni feature riduce l'incertezza sulla classe

Soglia: $IG \geq 0,01$

Bookkeeper : 11→9 | Storm: 11 → 6

Confronto dei modelli predittivi

Apache Bookkeeper

Algoritmo	CompositeScore	AUC	Precision	Recall	F1-Score	Kappa	NPofB20	Accuracy
Random Forest	0.635	0.843	0.859	0.748	0.798	0.612	0.582	0.862
IBk	0.634	0.840	0.856	0.748	0.796	0.611	0.582	0.860
Naive Bayes	0.559	0.729	0.840	0.660	0.737	0.540	0.541	0.825

Apache Storm

Algoritmo	CompositeScore	AUC	Precision	Recall	F1-Score	Kappa	NPofB20	Accuracy
Random Forest	0.579	0.956	0.795	0.516	0.625	0.583	0.659	0.9215
IBk	0.568	0.951	0.759	0.482	0.590	0.545	0.629	0.914
Naive Bayes	0.450	0.828	0.464	0.339	0.391	0.317	0.440	0.8654

RQ1: Risultati Predittivi

Random Forest è il classificatore più efficace



Random Forest

Bookkeeper: Score 0.635

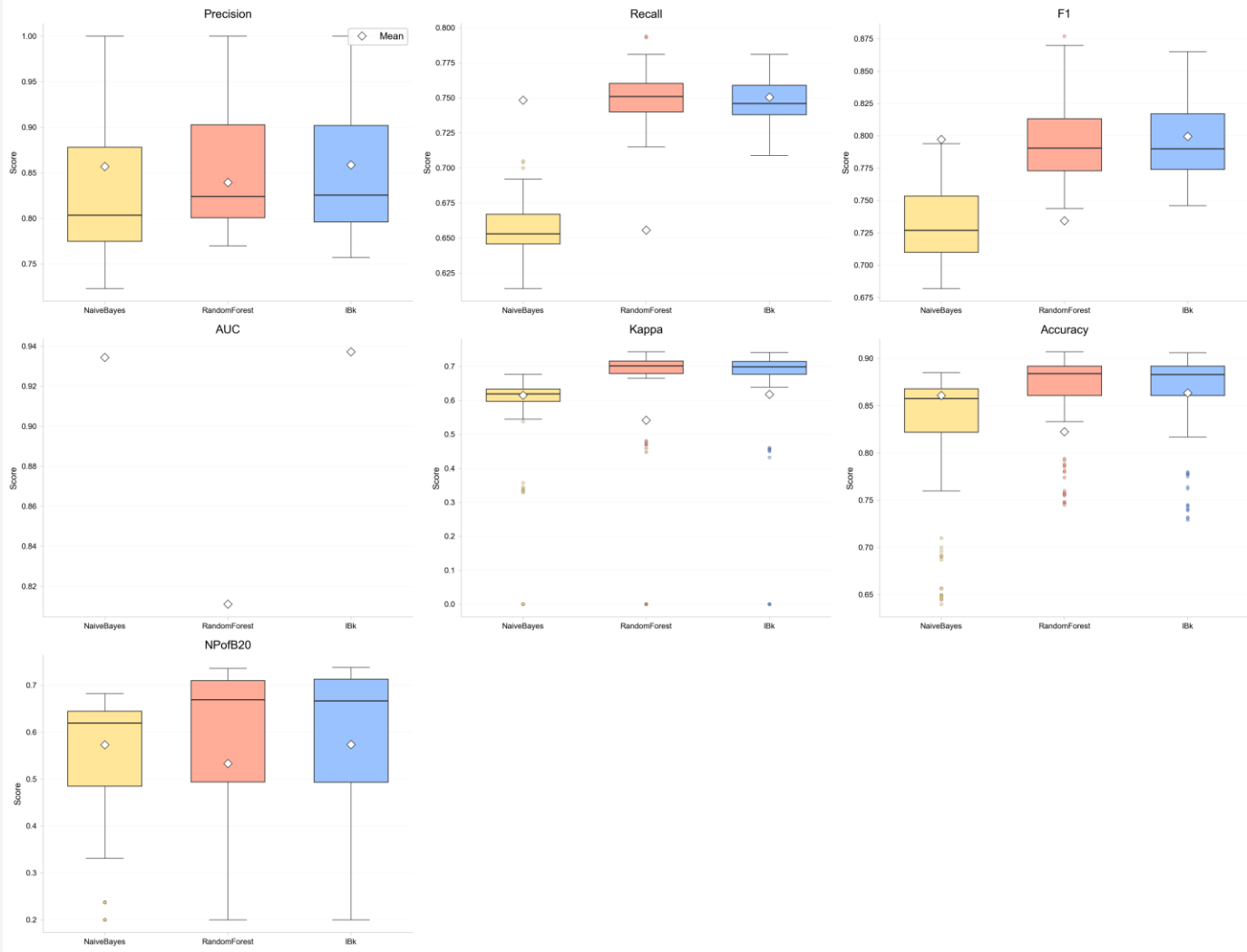
Storm: Score 0.579

Migliore bilanciamento tra
precisione e recall

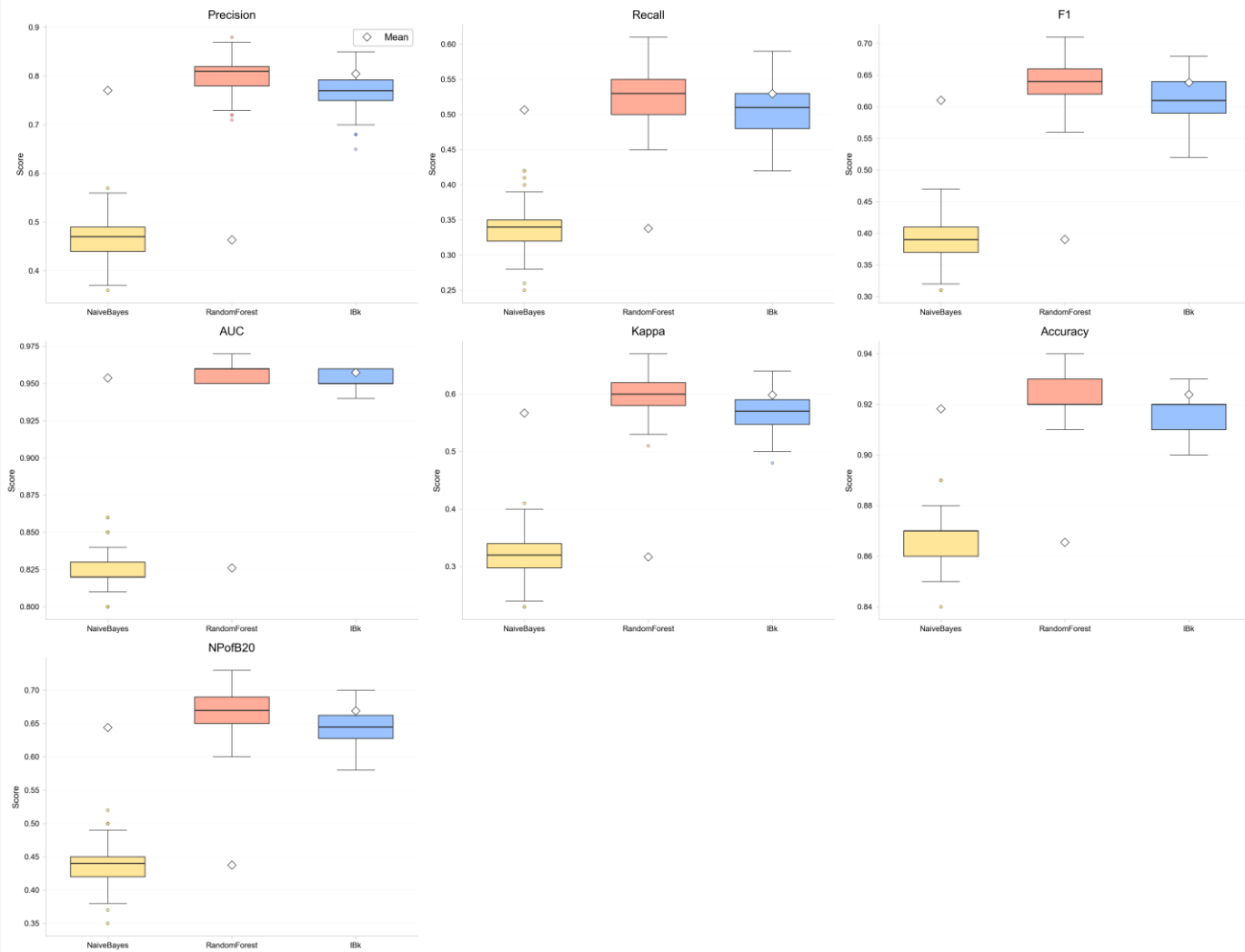
IBk: Performance comparabile, secondo posto

Naive Bayes: Recall significativamente inferiore

BOOKKEEPER Project - Complete Performance Metrics Overview



STORM Project - Complete Performance Metrics Overview



Refactoring: Apache Bookkeeper

Metodo: processPacket(ByteBuffer, Cnxn) in BookieServer.java

✗ Prima

Problema: Metodo monolitico che viola il principio di singola responsabilità

Metodo	processPacket (ByteBuffer,Cnxn)
Versione	4.2.1
LOC	174
ParameterCount	2
CycloComplexity	33
CognitiveComplexity	73
NestingDepth	5
Revisions	89
Authors	30
TotalChurn	1909
MaxChurn	111
AvgChurn	11.42
NumberOfCodeSmells	14
Bugginess	yes

✓ Dopo

Tecnica: Extract Method

Classificazione: Non Buggy

Metodo	processPacket (ByteBuffer,Cnxn)
Versione	4.2.1
LOC	29
ParameterCount	2
CycloComplexity	6
CognitiveComplexity	4
NestingDepth	2
Revisions	89
Authors	30
TotalChurn	1909
MaxChurn	111
AvgChurn	11.42
NumberOfCodeSmells	0
Bugginess	no

Refactoring: Apache Storm

Metodo: select(String, List) in JdbcClient.java

✗ Prima

Problema: Strutture di controllo eccessivamente annidate

Metodo	Select(String,List)
Versione	0.10.2
LOC	54
ParameterCount	2
CycloComplexity	16
CognitiveComplexity	10
NestingDepth	14
Revisions	33
Authors	16
TotalChurn	497
MaxChurn	74
AvgChurn	8.59
NumberOfCodeSmells	3
Bugginess	yes

✓ Dopo

Tecnica: Extract Method
Intera classe: Non Buggy

Metodo	Select(String,List)
Versione	0.10.2
LOC	19
ParameterCount	2
CycloComplexity	4
CognitiveComplexity	6
NestingDepth	3
Revisions	33
Authors	16
TotalChurn	497
MaxChurn	74
AvgChurn	8.59
NumberOfCodeSmells	0
Bugginess	no

Analisi What-If

Apache Bookkeeper

Dataset	Actual Buggy	Estimated Buggy
A	4838	4220
B+	2158	2034
B	-	1846
C	2680	2186

Apache Storm

Dataset	Actual Buggy	Estimated Buggy
A	12611	7860
B+	5368	3513
B	-	3452
C	7243	4347

- B+ porzione di A con Afeature > 0
- C porzione di A con Afeature = 0
- B B+ manipolato con Afeature settata a 0

RQ2: Analisi What-If

Se avessimo rifattorizzato preventivamente...

Apache Bookkeeper

188 metodi buggy in meno

≈ 9.2% di riduzione • Forte impatto delle metriche strutturali

Apache Storm

61 metodi buggy in meno

≈ 1.7% di riduzione • Dominanza delle metriche storiche

Conclusioni



1

Il refactoring preventivo funziona

Interventi mirati possono ridurre significativamente i metodi buggy, ma l'efficacia dipende dal tipo di progetto

2

Le metriche storiche dominano

Revisions e Authors sono i predittori più forti, ma le metriche strutturali sono quelle su cui possiamo agire

3

Progetti diversi, strategie diverse

Bookkeeper beneficia di refactoring strutturale, Storm richiede attenzione al processo di sviluppo

4

Random Forest è il vincitore

Migliore bilanciamento tra precisione e recall per la predizione di difetti a livello di metodo

Sviluppi Futuri

- Estendere l'analisi ad altri progetti open source per validare la generalizzabilità dei risultati
- Includere metriche più specifiche come accoppiamento e coesione tra componenti
- Sviluppare strumenti automatici per misurazioni più accurate



https://github.com/tostfia/ISW2_project.git



https://sonarcloud.io/project/overview?id=tostfia_ISW2_project

Grazie per l'attenzione

Sofia Tosti
sofia.tosti@students.uniroma2.eu
Università di Roma Tor Vergata